

AI DÀNH CHO LẬP TRÌNH VIÊN

Cẩm Nang Nhập Môn

1. BỨC TRANH TOÀN CẢNH VỀ AI (The Big Picture)

AI là một lĩnh vực rất rộng. Là lập trình viên, việc hiểu rõ các khái niệm cốt lõi liên kết với nhau như thế nào là điều rất quan trọng.

★ Trí tuệ nhân tạo (AI):

Khả năng của máy móc trong việc mô phỏng trí thông minh con người và thực hiện các tác vụ vốn đòi hỏi trí tuệ con người.

★ Học máy (ML):

Một nhánh của AI, trong đó máy móc học các quy luật từ dữ liệu và tự cải thiện mà không cần lập trình tường minh.

★ Học sâu (DL):

Một nhánh của ML sử dụng mạng nơ-ron với nhiều lớp để học các quy luật phức tạp từ lượng lớn dữ liệu.

★ AI Tạo sinh (GenAI):

Một dạng AI có thể tạo ra nội dung mới (văn bản, hình ảnh, mã nguồn, âm thanh, video, v.v.) dựa trên những gì đã học được.

★ AI Tác tử (Agentic AI):

Các hệ thống AI có khả năng suy nghĩ, lập kế hoạch, hành động bằng công cụ, và đạt được mục tiêu với sự can thiệp tối thiểu của con người.

Mối Liên Hệ Giữa Các Khái Niệm

Trí tuệ nhân tạo (AI)

Mô phỏng trí thông minh con người

Học máy (ML)

Học từ dữ liệu

Học sâu (DL)

Dùng mạng nơ-ron (nhiều lớp)

AI Tạo sinh (GenAI)

Tạo ra nội dung mới

AI Tác tử

Lập kế hoạch, hành động và đạt mục tiêu một cách tự chủ

↑ Khái niệm rộng hơn
↓ Cụ thể & nâng cao hơn

Bảng So Sánh Nhanh

Khái niệm	Trọng tâm	Ví dụ	Ứng dụng
AI	Mô phỏng trí thông minh con người	Chatbot, xe tự lái	Giải quyết vấn đề tổng quát
ML	Học từ dữ liệu	Phát hiện spam, hệ gợi ý	Học quy luật & dự đoán
DL	Học bằng mạng nơ-ron sâu	Nhận diện ảnh, nhận diện giọng nói	Dữ liệu phức tạp (ảnh, âm thanh...)
GenAI	Tạo nội dung mới	ChatGPT, DALL-E, GitHub Copilot	Tạo nội dung, tự động hóa
Agentic AI	Hành động tự chủ để đạt mục tiêu	AutoGPT, các AI Agent dùng công cụ	Tự động hóa tác vụ, ra quyết định

2. CÁCH LLM HOẠT ĐỘNG

LLM (Mô hình ngôn ngữ lớn) là "bộ não" đứng sau hầu hết các công cụ AI như ChatGPT, Claude, Gemini, v.v. Chúng hiểu văn bản và tạo ra phản hồi giống con người.

CÁCH LLM SINH RA TOKEN TIẾP THEO



LLM LÀ GÌ?

LLM là một mô hình được huấn luyện trên lượng dữ liệu văn bản khổng lồ. Nó học các quy luật trong văn bản và dùng những quy luật đó để dự đoán từ (hoặc token) tiếp theo trong một chuỗi.

Ý CHÍNH

LLM không "biết" sự thật. Nó dự đoán token tiếp theo dựa trên các quy luật đã thấy trong quá trình huấn luyện.

THÀNH PHẦN CHÍNH

- Dữ liệu huấn luyện:** Mô hình học từ hàng tỷ từ trên internet, sách, bài viết, mã nguồn, v.v.
- Token:** Văn bản được chia thành các mảnh nhỏ gọi là token.
- Mô hình (Transformer):** Dùng kiến trúc transformer để tìm quy luật và mối quan hệ giữa các token.
- Suy luận (Inference):** Khi nhận một prompt, mô hình dự đoán từng token một để tạo ra phản hồi.

QUY TRÌNH HUẤN LUYỆN (TÓM TẮT)

1. Thu thập lượng lớn dữ liệu văn bản

↓

2. Học quy luật (Dự đoán từ tiếp theo)

↓

3. Mô hình lớn (Kiến trúc Transformer)

↓

4. Mô hình sẵn sàng để suy luận (sử dụng)

VÍ DỤ (DỰ ĐOÁN TOKEN TIẾP THEO)

Prompt: The sky is (Bầu trời thì)

Mô hình dự đoán các token tiếp theo:

Bước 1 → xanh

Bước 2 → và

Bước 3 → trong

Bước 4 → với

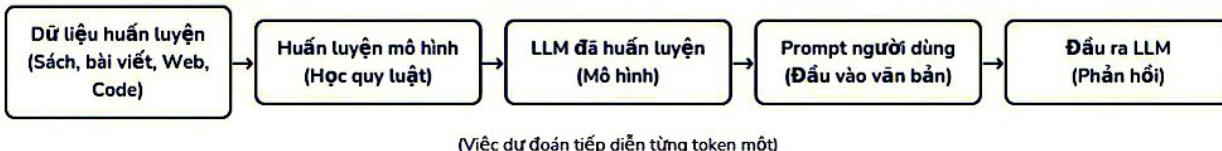
Bước 5 → mây

Ở mỗi bước, mô hình chọn token khả dĩ nhất tiếp theo và tiếp tục quá trình sinh văn bản.

ĐIỂM QUAN TRỌNG

- LLM tạo ra văn bản, không phải sự thật.
- Chúng ghi nhớ quy luật, không phải thông tin cụ thể.
- Càng nhiều dữ liệu và huấn luyện tốt hơn, hiệu suất càng cao.
- LLM là nền tảng cho các công cụ như ChatGPT, Copilot, v.v.

GÓC NHÌN ĐƠN GIẢN VỀ QUY TRÌNH LLM



3. KỸ THUẬT PROMPT

Kỹ thuật prompt là nghệ thuật đặt câu hỏi đúng cách để LLM đưa ra phản hồi chính xác, hữu ích và nhất quán.

Ý CHÍNH

Prompt tốt hơn
=
Phản hồi tốt hơn

CÁC LOẠI PROMPT

- Zero-shot Prompting:** Hỏi trực tiếp mà không có ví dụ nào. (Tốt cho câu hỏi đơn giản)
- One-shot Prompting:** Đưa ra một ví dụ rồi hỏi. (Tốt hơn cho sự rõ ràng)
- Few-shot Prompting:** Đưa ra nhiều ví dụ rồi hỏi. (Tốt nhất cho độ chính xác)
- Chain-of-Thought (CoT):** Yêu cầu mô hình suy nghĩ từng bước trước khi trả lời.
- Role Prompting:** Gán một vai trò cho mô hình (VD: Bạn là chuyên gia Python)
- Context Prompting:** Cung cấp bối cảnh để mô hình hiểu tình huống.

MẪU PROMPT (DÙNG ĐỊNH DẠNG NÀY)

Role:	Định nghĩa mô hình là ai.
Task:	Điều bạn muốn mô hình làm.
Context:	Cung cấp thông tin bối cảnh.
Rules:	Đưa ra các quy tắc/ràng buộc quan trọng.
Format:	Câu trả lời nên được trình bày ra sao.
Input:	Dữ liệu hoặc câu hỏi thực tế.
Output:	Phản hồi mong đợi.

PROMPT TỆ vs PROMPT TỐT

PROMPT TỆ	PROMPT TỐT
Giải thích về list trong Python. ↓ (Quá mơ hồ) Phản hồi → Chung chung, ngắn gọn, không hữu ích.	Bạn là chuyên gia Python. Hãy giải thích về list trong Python kèm ví dụ. Bao gồm các phương thức, phép toán và một đoạn code ngắn minh họa. ↓ (Rõ ràng & cụ thể) Phản hồi → Chi tiết, có cấu trúc và hữu ích.

VÍ DỤ CHAIN OF THOUGHT (CoT)

Prompt:
Hỏi: Nếu một đoàn tàu đi 60 km trong 1 giờ, nó sẽ đi được bao xa trong 4,5 giờ?
Hãy suy nghĩ từng bước.

Suy luận từng bước:
Bước 1 → Tốc độ = 60 km / 1 giờ = 60 km/h
Bước 2 → Thời gian = 4,5 giờ
Bước 3 → Quãng đường = Tốc độ × Thời gian
Bước 4 → Quãng đường = 60 × 4,5 = 270 km
Đáp án: 270 km

MẸO ĐỂ CÓ PROMPT TỐT HƠN

- Rõ ràng và cụ thể.
- Cung cấp đủ bối cảnh.
- Chia nhỏ tác vụ phức tạp thành các bước.
- Đưa ví dụ (nếu cần).
- Nói rõ định dạng câu trả lời.
- Yêu cầu mô hình suy nghĩ từng bước.
- Lặp lại và cải thiện prompt.

Chỉ cần thay đổi nhỏ trong prompt cũng có thể tạo ra thay đổi lớn trong kết quả.

4. RAG (SINH VĂN BẢN TĂNG CƯỜNG BẰNG TRUY XUẤT)

RAG giúp LLM sử dụng dữ liệu riêng của bạn để đưa ra câu trả lời chính xác và **cập nhật**.

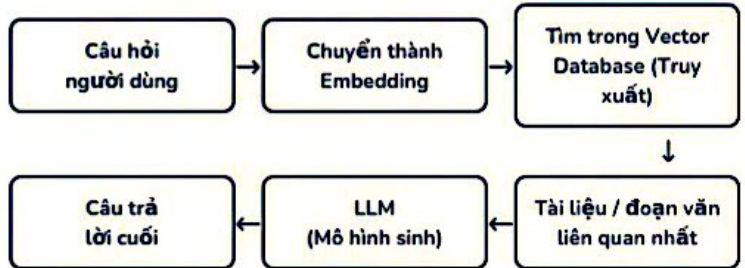
TẠI SAO CẦN RAG?

- LLM có giới hạn về thời điểm dữ liệu huấn luyện (knowledge cutoff).
- Chúng có thể "ảo giác" (bịa ra thông tin).
- RAG cung cấp thông tin thực tế, chuyên biệt theo lĩnh vực.

RAG LÀ GÌ?

RAG kết hợp việc **truy xuất** thông tin liên quan từ một nguồn tri thức với sức mạnh sinh văn bản của LLM để tạo ra câu trả lời chính xác.

CÁCH RAG HOẠT ĐỘNG (QUY TRÌNH)



Tóm gọn:

Truy xuất thông tin liên quan → Đưa cho LLM → LLM tạo câu trả lời tốt hơn.

GIẢI THÍCH QUY TRÌNH RAG

- ➊ **Nạp dữ liệu (Ingestion):** Tải tài liệu (pdf, txt, docs, trang web, v.v.) và chia thành các đoạn nhỏ.
- ➋ **Embedding:** Chuyển mỗi đoạn thành vector (dãy số). Lưu vào Vector Database.
- ➌ **Truy xuất (Retrieval):** Người dùng hỏi → chuyển thành vector → tìm các đoạn tương tự.
- ➍ **Tăng cường (Augmentation):** Gửi các đoạn đã truy xuất + câu hỏi cho LLM.
- ➎ **Sinh câu trả lời (Generation):** LLM dùng bối cảnh đó để tạo câu trả lời chính xác.

RAG vs LLM THÔNG THƯỜNG

LLM thông thường

- Chỉ dùng dữ liệu huấn luyện.
- Có thể đã lỗi thời.
- Có thể ảo giác.
- Không truy cập được tài liệu riêng của bạn.

RAG

- Dùng dữ liệu riêng của bạn.
- Chính xác hơn.
- Ít ảo giác hơn.
- Luôn cập nhật theo dữ liệu của bạn.

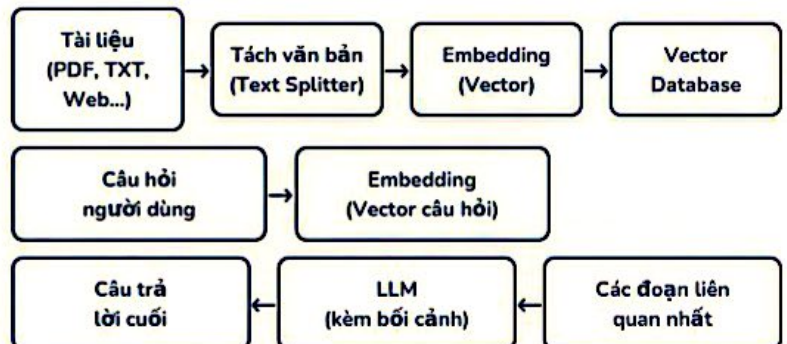
RAG ĐƯỢC DÙNG Ở ĐÂU?

- Trò chuyện với PDF / tài liệu
- Chatbot tri thức nội bộ doanh nghiệp
- Chatbot hỗ trợ khách hàng
- Trợ lý nghiên cứu
- Pháp lý, tài chính, y tế, giáo dục, v.v.

MẸO

Chất lượng của RAG phụ thuộc vào chất lượng dữ liệu, kích thước đoạn (chunk) và embedding của bạn.

KIẾN TRÚC RAG ĐƠN GIẢN



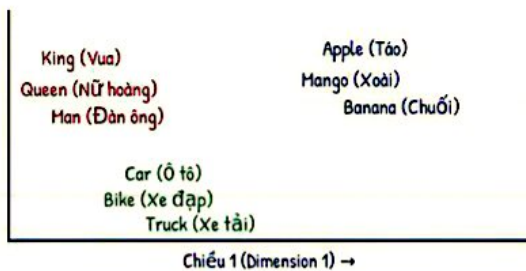
5. EMBEDDING & VECTOR DATABASE

EMBEDDING LÀ GÌ?

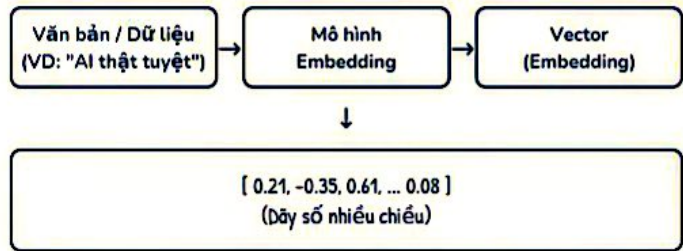
Embedding là biểu diễn số học (vector) của văn bản, hình ảnh hoặc dữ liệu trong không gian nhiều chiều, nơi các mục có ý nghĩa tương tự được đặt gần nhau.

TẠI SAO CẦN EMBEDDING?

- Máy móc hiểu số, không hiểu từ ngữ.
- Nắm bắt ý nghĩa / ngữ cảnh của văn bản.
- Hỗ trợ tìm kiếm tương tự (similarity search).
- Hữu ích cho RAG, tìm kiếm, gợi ý, v.v.



CÁCH EMBEDDING HOẠT ĐỘNG (TỔNG QUAN)

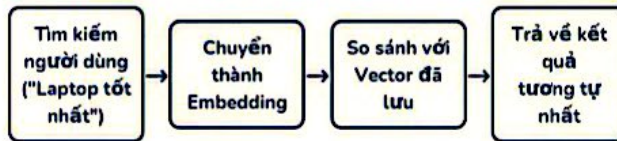


Ý CHÍNH:

Ý nghĩa tương tự → Vector gần nhau
Ý nghĩa khác nhau → Vector xa nhau

Khoảng cách được tính bằng các phép đo như Cosine Similarity (**Độ tương đồng Cosine**).

TÌM KIẾM TƯƠNG TỰ (DÙNG EMBEDDING)



VECTOR DATABASE LÀ GÌ?

Vector database là loại cơ sở dữ liệu đặc biệt lưu trữ embedding và cho phép tìm kiếm tương tự nhanh trên các vector nhiều chiều.

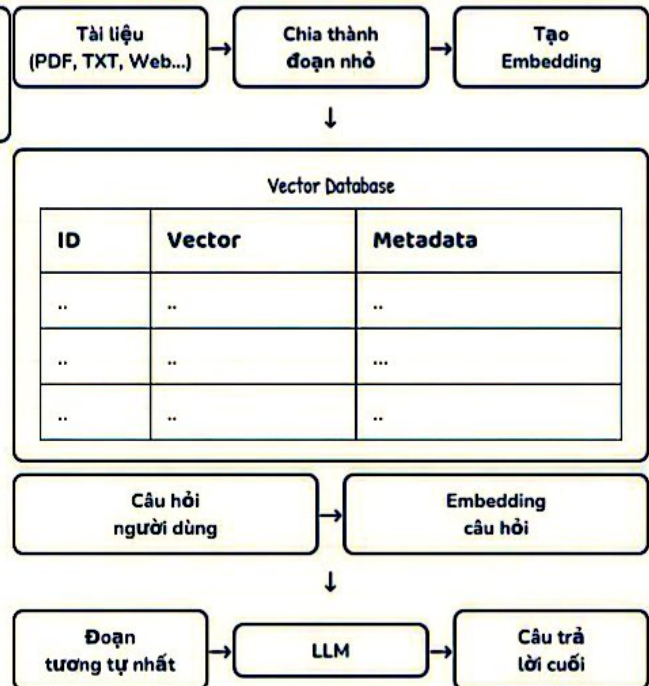
Ví dụ:

Pinecone · Chroma · Weaviate · FAISS · Milvus

Vector DB lưu trữ:

- Vector (embedding)
 - Văn bản / dữ liệu gốc
 - Metadata (id, nguồn, số trang, v.v.)
- Được tối ưu cho tìm kiếm tương tự, không phải khớp chính xác.

CÁCH VECTOR DATABASE HOẠT ĐỘNG



EMBEDDING + VECTOR DB = SỰ KẾT HỢP MẠNH MẼ

- Tìm kiếm ngữ nghĩa (dựa trên ý nghĩa)
- RAG (Sinh văn bản tăng cường bằng truy xuất)
- Trò chuyện với dữ liệu riêng của bạn
- Gợi ý, phân cụm, phân loại, v.v.

MẸO:

Mô hình embedding tốt + chia đoạn tốt + Vector DB tốt = Kết quả chất lượng cao.

6. AI AGENT & AGENTIC AI

AI AGENT LÀ GÌ?

AI Agent là một hệ thống có thể **cảm nhận** môi trường, suy nghĩ, thực hiện hành **động** và đạt được **mục tiêu** thay cho người dùng.

Nó có thể sử dụng công cụ, ghi nhớ và cải thiện quyết định của mình.

CÁC LOẠI (CẤP ĐỘ)

① **AI Assistant (Trợ lý AI)**: Trả lời câu hỏi, hỗ trợ tác vụ. (VD: ChatGPT)

② **AI Agent**: Có thể dùng công cụ, truy cập dữ liệu, thực hiện tác vụ.

③ **Agentic AI**: Tự chủ, lập kế hoạch, học hỏi và ra quyết định để đạt mục tiêu phức tạp.

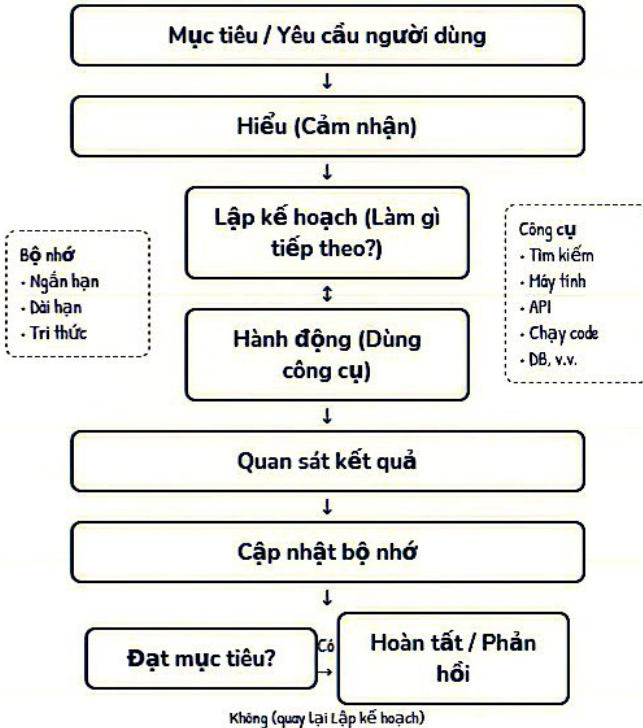
KHÁC BIỆT CHÍNH

AI Assistant → Phản hồi
AI Agent → Hành động
Agentic AI → Suy nghĩ, Lập kế hoạch, Hành động, Cải thiện

NĂNG LỰC CỐT LÕI CỦA AI AGENT

- Tự chủ (Autonomy)**: Hoạt động với sự trợ giúp tối thiểu của con người.
- Suy luận (Reasoning)**: Hiểu và suy luận về các tác vụ.
- Lập kế hoạch (Planning)**: Chia nhỏ mục tiêu lớn thành các bước.
- Dùng công cụ (Tool Use)**: Sử dụng công cụ/API bên ngoài.
- Bộ nhớ (Memory)**: Ghi nhớ ngữ cảnh và tương tác trước đó.
- Học hỏi (Learning)**: Cải thiện từ phản hồi và kinh nghiệm.

CÁCH AI AGENT HOẠT ĐỘNG (QUY TRÌNH)



VÍ DỤ: AI AGENT (LẬP KẾ HOẠCH DU LỊCH)

Mục tiêu: Lên kế hoạch chuyến đi từ Hà Nội đến Đà Nẵng vào cuối tuần tới

Bước	Agent làm gì
1. Hiểu	Hiểu địa điểm, ngày, ngân sách
2. Lập kế hoạch	Chia nhỏ: chuyến bay, chỗ ở, địa điểm, v.v.
3. Hành động	Tìm chuyến bay, khách sạn, thời tiết bằng công cụ
4. Quan sát	Thu thập kết quả từ các công cụ
5. Quyết định	Chọn phương án tốt nhất
6. Phản hồi	Đưa kế hoạch du lịch cuối cùng cho người dùng

KIẾN TRÚC AGENTIC AI (TỔNG QUAN CẤP CAO)



MẸO HAY

Hãy bắt đầu với một quy trình đơn giản trước khi thêm nhiều công cụ và bộ nhớ phức tạp.

ĐIỂM QUAN TRỌNG

★ Agent hoạt động theo vòng lặp cho đến khi đạt được mục tiêu.

7. MCP (GIAO THỨC NGŨ CẢNH MÔ HÌNH)

MCP LÀ GÌ?

MCP là một giao thức mở cho phép các mô hình AI kết nối với công cụ, nguồn dữ liệu và hệ thống bên ngoài theo cách chuẩn hóa.

TẠI SAO CẦN MCP?

- Giúp mô hình AI hữu ích và mạnh mẽ hơn.
- Đễ dàng tích hợp với công cụ và dữ liệu.
- Truy cập an toàn và có kiểm soát.
- Hoạt động trên nhiều nền tảng và mô hình khác nhau.

THÀNH PHẦN CỦA MCP

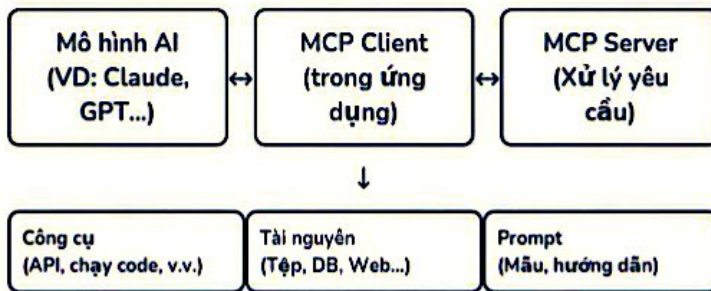
- MCP Host / Client:** Ứng dụng dùng MCP để kết nối với server.
- MCP Server:** Cung cấp công cụ, tài nguyên và prompt qua MCP.
- Công cụ (Tools):** Các hàm mà AI có thể gọi (VD: Tìm kiếm, máy tính, chạy code).
- Tài nguyên (Resources):** Nguồn dữ liệu như tệp, DB, API, tài liệu.
- Prompt:** Các mẫu prompt tái sử dụng cho tương tác nhất quán.

Tích hợp truyền thống	MCP
Tích hợp tùy chỉnh	Giao thức chuẩn hóa
Khó mở rộng	Đễ thêm công cụ mới
Khác nhau cho mỗi ứng dụng	Hoạt động trên mọi ứng dụng
Nhiều code & logic hơn	Đơn giản & tái sử dụng

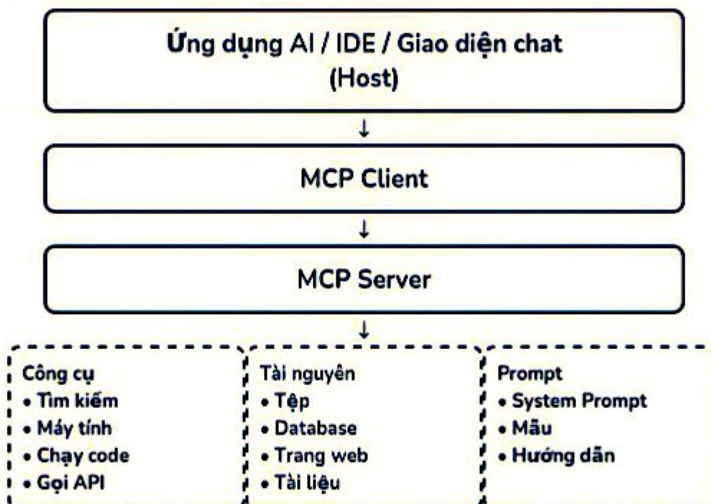
LỢI ÍCH CỦA MCP

- Cách chuẩn để kết nối AI với thế giới bên ngoài.
- Giảm thiểu code và độ phức tạp tùy chỉnh.
- Đễ mở rộng với công cụ và nguồn dữ liệu mới.
- Cải thiện bảo mật và kiểm soát.
- Bền vững theo thời gian và không phụ thuộc mô hình.

CÁCH MCP HOẠT ĐỘNG (TỔNG QUAN)



KIẾN TRÚC MCP (SƠ ĐỒ)



VÍ DỤ THỰC TẾ

Bạn đang dùng một trợ lý lập trình AI trong IDE. Thông qua MCP, nó có thể:

- Đọc các tệp trong dự án của bạn
- Tìm kiếm tài liệu
- Chạy test
- Triển khai code
- Tất cả mà không cần rời khỏi IDE!

TÓM TẮT MCP

MCP giống như một bộ kết nối vạn năng giữa các mô hình AI và công cụ / dữ liệu / hệ thống mà chúng cần để thực hiện công việc thực tế.

8. TÍNH CHỈNH & THÍCH ỨNG MÔ HÌNH

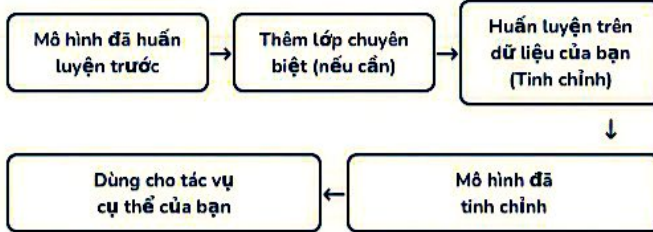
TÍNH CHỈNH (FINE-TUNING) LÀ GÌ?

Fine-tuning là việc huấn luyện thêm một mô hình đã được huấn luyện trước (pre-trained) trên dữ liệu cụ thể để nó hoạt động tốt hơn trong một lĩnh vực hoặc tác vụ riêng.

TẠI SAO CẦN TÍNH CHỈNH?

- Mô hình đã huấn luyện trước mang tính tổng quát.
- Tính chỉnh giúp chúng trở nên chuyên biệt và chính xác.
- Giúp cải thiện kết quả cho trường hợp sử dụng của bạn.
- Cần ít dữ liệu hơn so với huấn luyện từ đầu.

CÁCH TÍNH CHỈNH HOẠT ĐỘNG (QUY TRÌNH)

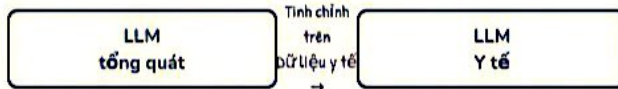


CÁC PHƯƠNG PHÁP TÍNH CHỈNH

- Full Fine-tuning (Tính chỉnh toàn phần):** Cập nhật toàn bộ tham số của mô hình. (Độ chính xác cao, tốn nhiều thời gian huấn luyện hơn)
- Feature-based Fine-tuning:** Đóng băng một số lớp, chỉ huấn luyện vài lớp cuối. (Thời gian huấn luyện ít hơn, kết quả tốt)
- Prompt-based Fine-tuning (PEFT):** Dùng các kỹ thuật như LoRA, Adapter, Prefix-tuning. (Rất hiệu quả, tốn ít bộ nhớ)

VÍ DỤ

Bạn có một LLM tổng quát. Bạn tính chỉnh nó trên dữ liệu y tế. Giờ nó có thể trả lời các câu hỏi liên quan đến y tế tốt hơn nhiều.



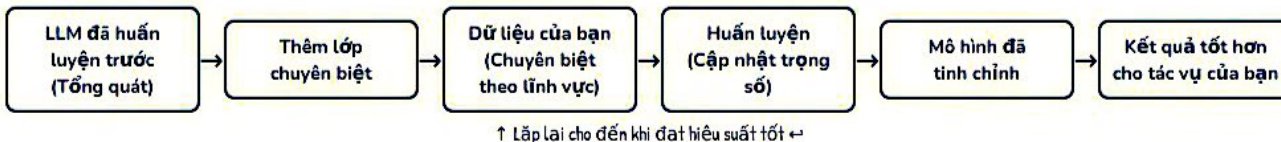
TRƯỜNG HỢP SỬ DỤNG PHỔ BIẾN

- Chatbot hỗ trợ khách hàng
- Phân tích tài liệu pháp lý
- Trợ lý y tế
- Sinh code cho ngôn ngữ chuyên biệt
- Hỏi đáp theo lĩnh vực chuyên biệt

FULL FINE-TUNING vs PEFT (VD: LoRA)

Đặc điểm	Full Fine-tuning	PEFT / LoRA
Tham số cập nhật	Tất cả	Ít (Adapter)
Dữ liệu cần	Nhiều	Ít
Thời gian huấn luyện	Nhiều	Ít
Bộ nhớ sử dụng	Cao	Thấp
Hiệu suất	Cao	Tốt (Hiệu quả)
Chi phí	Cao	Thấp

SƠ ĐỒ ĐƠN GIẢN - TÍNH CHỈNH (VÍ DỤ)



MẸO

Bắt đầu với PEFT (như LoRA). Nếu cần hiệu suất cao hơn, hãy chuyển sang Full Fine-tuning.

ĐIỂM QUAN TRỌNG

- ★ Dữ liệu tốt + mô hình tốt + tính chỉnh tốt = kết quả tốt nhất.
- ★ Chất lượng dữ liệu quan trọng hơn số lượng.
- ★ Luôn đánh giá mô hình trên dữ liệu thực tế.

9. NGĂN XẾP CÔNG NGHỆ AI CHO LẬP TRÌNH VIÊN

AI DEVELOPER STACK LÀ GÌ?

AI Developer Stack là sự kết hợp giữa mô hình, công cụ, framework và dịch vụ giúp lập trình viên xây dựng ứng dụng chạy trên nền tảng AI.

TẠI SAO STACK QUAN TRỌNG?

- Chọn đúng mô hình cho đúng tác vụ.
- Hiệu suất và chi phí tốt hơn.
- Tích hợp và mở rộng dễ dàng.
- Phát triển và triển khai nhanh hơn.

CÁC THÀNH PHẦN CHÍNH TRONG STACK

- Mô hình (Models):** Chọn LLM phù hợp cho trường hợp sử dụng của bạn.
- Framework điều phối (Orchestration):** Quản lý prompt, chuỗi xử lý, bộ nhớ, agent.
- Công cụ & API:** Bổ sung năng lực bên ngoài (tìm kiếm, chạy code, truy cập DB, xử lý tệp).
- Tầng dữ liệu (Data Layer):** Lưu trữ và truy xuất tri thức hiệu quả.
- Tầng triển khai (Deployment):** Triển khai và mở rộng ứng dụng AI của bạn.
- Tầng ứng dụng (Application):** Frontend / backend mà người dùng tương tác.

MÔ HÌNH AI (FOUNDATION MODELS)

Nhà cung cấp	Dòng mô hình
OpenAI	GPT-4o, GPT-4 Turbo, GPT-3.5
Anthropic	Claude Opus, Sonnet, Haiku
Google (Gemini)	Gemini Pro, Gemini Flash
Meta (Llama)	Llama 3, Llama 3.1
Mistral AI	Mixtral 8x22B, Mistral Large
Cohere	Command R+, Command R
AI21 Labs	Jurassic-2
DeepSeek	DeepSeek-V2, DeepSeek-Coder

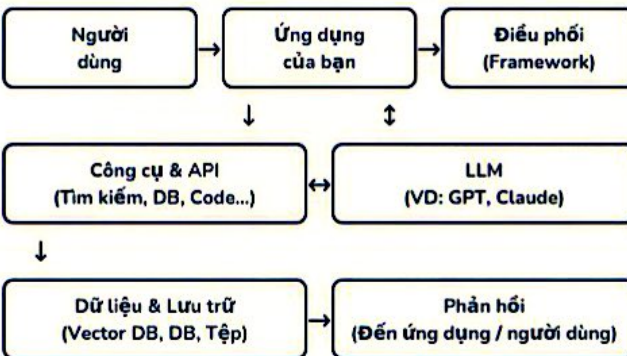
DỊCH VỤ & NỀN TẢNG PHỔ BIẾN

- OpenAI API (mô hình GPT)
- Anthropic API (mô hình Claude)
- Google Vertex AI (mô hình Gemini)
- Hugging Face (mô hình & suy luận)
- Cohere API
- Mistral AI API
- Replicate (chạy mô hình mã nguồn mở)
- Groq (suy luận tốc độ cao)
- Pinecone (Vector Database)
- Supabase / Firebase (Backend as a Service)

CÁC TẦNG TRONG STACK (TỔNG QUAN)

1. MÔ HÌNH	Mô hình nền tảng (LLM) như GPT, Claude, Gemini, Llama
2. ĐIỀU PHỐI	Framework kết nối mô hình, công cụ và bộ nhớ (LangChain, v.v.)
3. CÔNG CỤ & API	API và công cụ tăng cường năng lực (tìm kiếm, DB, code, v.v.)
4. DỮ LIỆU & LƯU TRỮ	Vector DB, cơ sở dữ liệu và lưu trữ tri thức
5. TRIỂN KHAI	Lưu trữ, phục vụ và giám sát (Cloud, Docker, Kubernetes)
6. ỨNG DỤNG	UI / API / App (nơi người dùng tương tác)

QUY TRÌNH AI DEVELOPER STACK ĐƠN GIẢN



HẸO: Chọn đúng mô hình + công cụ + dữ liệu → Ứng dụng AI tốt hơn